

Trabajo Práctico Integrador - Laboratorio 2

Autor: Flores Gonzalo

Índice de Contenidos

A continuación se presenta la organización analítica del presente documento técnico:

- Marco Teórico:
 1. Paradigma de Programación Orientada a Objetos (POO)
 2. Lenguaje de Programación Java y Colecciones en Memoria
 3. Arquitectura de Software y Almacenamiento Centralizado
- Decisiones Técnicas y Arquitectura
 1. Organización Jerárquica de Paquetes
 2. Diseño del Modelo de Dominio y Relaciones de Entidad
 3. Interfaces y Encapsulamiento de Reglas de Negocio
- Interfaces de Usuario y Flujo de Ejecución
 1. Flujo del Menú Principal
- Dificultades Encontradas y Soluciones Aplicadas
 1. Persistencia Volátil y Sincronización Bidireccional
 2. Prevención de Excepciones en Entradas de Consola
- Recursos, Enlaces y Bibliografía
 1. Enlaces de Repositorio y Video Explicativo

Marco Teórico

Funcionalmente, el proyecto busca solucionar la falta de un sistema centralizado y automatizado para la gestión operativa diaria de un negocio gastronómico (**FoodStore**).

En concreto, el aplicativo resuelve los siguientes problemas del negocio:

- **Centralización del Catálogo:** Permite organizar de manera clara la oferta comercial a través de la creación y edición de categorías (como Bebidas, Comidas, Postres) y productos (con sus respectivos precios, descripciones y control de stock).
- **Automatización de Ventas y Cálculos:** Elimina los errores manuales al registrar un pedido. El sistema calcula de forma automática los subtotales por producto y el monto total facturado de la orden basándose en las cantidades solicitadas.
- **Control de Stock Inmediato:** Valida en tiempo real si el local cuenta con la mercadería suficiente antes de confirmar el pedido, evitando vender productos que no están disponibles.
- **Trazabilidad de Clientes e Historial:** Asocia cada orden de compra a un usuario registrado, permitiendo conocer qué consumió cada cliente, bajo qué estado se encuentra su pedido (Pendiente, Confirmado, Terminado, Cancelado) y qué medio de pago utilizó.
- **Baja Lógica (Seguridad de Datos):** Permite "eliminar" registros (usuarios, productos, categorías) sin perder el historial financiero ni corromper las estadísticas de ventas de pedidos anteriores (mediante un sistema de borrado lógico).

1.1 Paradigma de Programación Orientada a Objetos (POO) El paradigma de POO modela el software basándose en "objetos" que combinan datos (atributos) y comportamiento (métodos). En el sistema implementado, se aplican estrictamente sus cuatro pilares:

- **Abstracción:** Se aislaron las propiedades esenciales de elementos del mundo real (como Usuario o Producto), omitiendo detalles irrelevantes para este contexto de negocio.
- **Encapsulamiento:** Se restringe el acceso directo a los atributos internos usando modificadores de acceso de tipo private, exponiendo el comportamiento seguro a través de métodos getters y setters validados.
- **Herencia:** Centralizada en la clase abstracta Base, la cual transfiere atributos estructurales homogéneos de auditoría e identidad a todas las entidades del dominio.
- **Polimorfismo:** Evidenciado en la implementación de contratos mediante la interfaz Calculable, permitiendo que múltiples objetos calculen sus estados financieros de acuerdo a su propia lógica interna.

1.2 Lenguaje de Programación Java debido a su sólida tipificación, portabilidad y ecosistema maduro. Al tratarse de un prototipo sin base de datos relacional activa instalada, el almacenamiento se resolvió mediante el framework de colecciones (Java Collections Framework), específicamente utilizando estructuras de tipo ArrayList para garantizar un acceso indexado rápido y dinámico durante el ciclo de vida de la ejecución de la consola

1.3 Arquitectura de Software y Almacenamiento Centralizado Para suplir de forma limpia una base de datos relacional tradicional, se adoptó la capa de servicios (clase DataStore). La cual asegura que exista una única instancia global de la base de datos en memoria para toda la aplicación, proveyendo un punto de acceso unificado y seguro para las operaciones CRUD (Crear, Leer, Actualizar, Borrar)

Decisiones Técnicas y Arquitectura

La arquitectura se diseñó buscando un bajo acoplamiento y una alta cohesión, separando taxativamente las responsabilidades de representación de datos, almacenamiento de negocio y control de la interfaz gráfica textual

2.1 Organización Jerárquica de Paquetes El código fuente se estructuró de manera estricta en los siguientes paquetes del sistema.

2.2 Diseño del Modelo de Dominio y Relaciones de Entidad Se destaca el uso de la superclase abstracta Base. Al heredar de ella, todas las tablas lógicas asumen un identificador numérico único (Long id), una marca temporal de creación automática (LocalDateTime createdAt) y un indicador booleano para borrado lógico (boolean eliminado). Esto emula fielmente el comportamiento de los frameworks ORM modernos (como Hibernate/JPA), evitando la pérdida de integridad referencial histórica en los pedidos cuando se da de baja un artículo o un cliente. Las relaciones entre entidades se modelaron de manera bidireccional controlada:

- Una Categoría mantiene una lista interna de objetos de tipo Producto.
- Cada Producto guarda una referencia directa a su Categoría contenedora.
- Un Pedido agrupa una colección de instancias de DetallePedido, componiendo la clásica estructura Cabecera-Detalle de los sistemas de facturación estándar.

2.3 Interfaces y Encapsulamiento de Reglas de Negocio El cálculo económico se realiza bajo demanda mediante el uso de la interfaz Calculable. La fórmula matemática para determinar el costo acumulado de un pedido se define mediante la sumatoria del producto de la cantidad por el precio unitario de cada ítem activo, expresado formalmente como:

Paquete (Package)	Responsabilidad Primaria	Clases Clave Incluidas
entities	Modelado de objetos de negocio y lógica de datos pura.	Base, Usuario, Producto, Categoria, Pedido, DetallePedido
enums	Definición de tipos constantes discretos del sistema.	Rol, Estado, FormaPago
interfaces	Contratos de comportamiento y abstracciones operativas.	Calculable
services	Motor de datos (Singleton) y asistentes globales de entrada.	DataStore, InputHelper
Menus	Vistas de consola y lógica de interacción con el usuario.	MenuCategorias, MenuProductos, MenuUsuarios, MenuPedidos
Ejecutable	Punto de entrada de la aplicación y	Main

Interfaces de Usuario y Flujo de Ejecución

El sistema interactúa de manera interactiva a través de la terminal estándar. Las capturas textuales simuladas a continuación reflejan el comportamiento exacto y validado de los diferentes menús operativos

3.1 Flujo del Menú Principal Al iniciar la aplicación mediante la clase Main, se disparan los métodos de inicialización de semillas en el DataStore y se despliega la interfaz base de selección de módulos:

```
-----  
                SISTEMA DE PEDIDOS  
-----  
1. Categorías  
2. Productos  
3. Usuarios  
4. Pedidos  
0. Salir  
-----  
Seleccione una opcion: _
```

3.2 Gestión Automatizada de Pedidos y Totales Cuando un operador accede al módulo de administración de órdenes de venta, el sistema lista los documentos vigentes con un desglose exhaustivo de totales económicos calculados en tiempo real, aplicando el filtrado automático de registros marcados como eliminados

```
--- Listado de Pedidos ---  
Pedido #1 | Fecha: 2026-06-22 | Estado: PENDIENTE | Pago: EFECTIVO  
Cliente: Gonza Flores (gonza@gmail.com)  
-> [Detalle #1] Cafe (Cantidad: 2) - Subtotal: $2400.0  
-> [Detalle #2] Torta de chocolate (Cantidad: 1) - Subtotal: $2500.0  
TOTAL DEL PEDIDO: $4900.0  
-----  
Pedido #2 | Fecha: 2026-06-22 | Estado: CONFIRMADO | Pago: TARJETA  
Cliente: Mayra Arroyo (mayra@gmail.com)  
-> [Detalle #1] Milanesa con papas (Cantidad: 2) - Subtotal: $10400.0  
TOTAL DEL PEDIDO: $10400.0  
-----
```

Dificultades Encontradas y Soluciones Aplicadas

Durante las fases de diseño y pruebas unitarias de integración modular, surgieron desafíos técnicos que requirieron la reformulación de ciertas estrategias de código:

Problema: Al modificar o reasignar la categoría de un producto en caliente dentro de `MenuProductos.editar()`, el objeto se indexaba en la nueva categoría, pero continuaba apareciendo listado dentro de la colección histórica de la categoría anterior, generando inconsistencias graves en el árbol de navegación.

Solución: Se modificó la lógica interna del mutador `setCategoria(Categoria categoria)` de la entidad `Producto`. Se incorporó una validación previa que comprueba si ya existía una categoría asignada de antemano; en caso afirmativo, se remueve el producto explícitamente de la lista antigua (`this.categoria.getProductos().remove(this)`) antes de establecer la nueva asociación jerárquica

Recursos y Enlaces

5. A los efectos de la validación práctica de las competencias evaluadas en la materia Programación 2, se adjuntan los accesos directos al código fuente completo y los materiales audiovisuales requeridos para la defensa del proyecto

Github:

https://github.com/FloresGonzaloAndres-dev/TP_Programacion2_Integrador

Video:

<https://youtu.be/Fhk2ooK3-kM>